

CORENTIN LUDWIG

Romain DUPONT DE POOTER

ARTHUR ARNOLD

PAUL CLUZEL

Ugo VALLAT

FILESYSTEM

CLB - 2025

SOMMAIRE

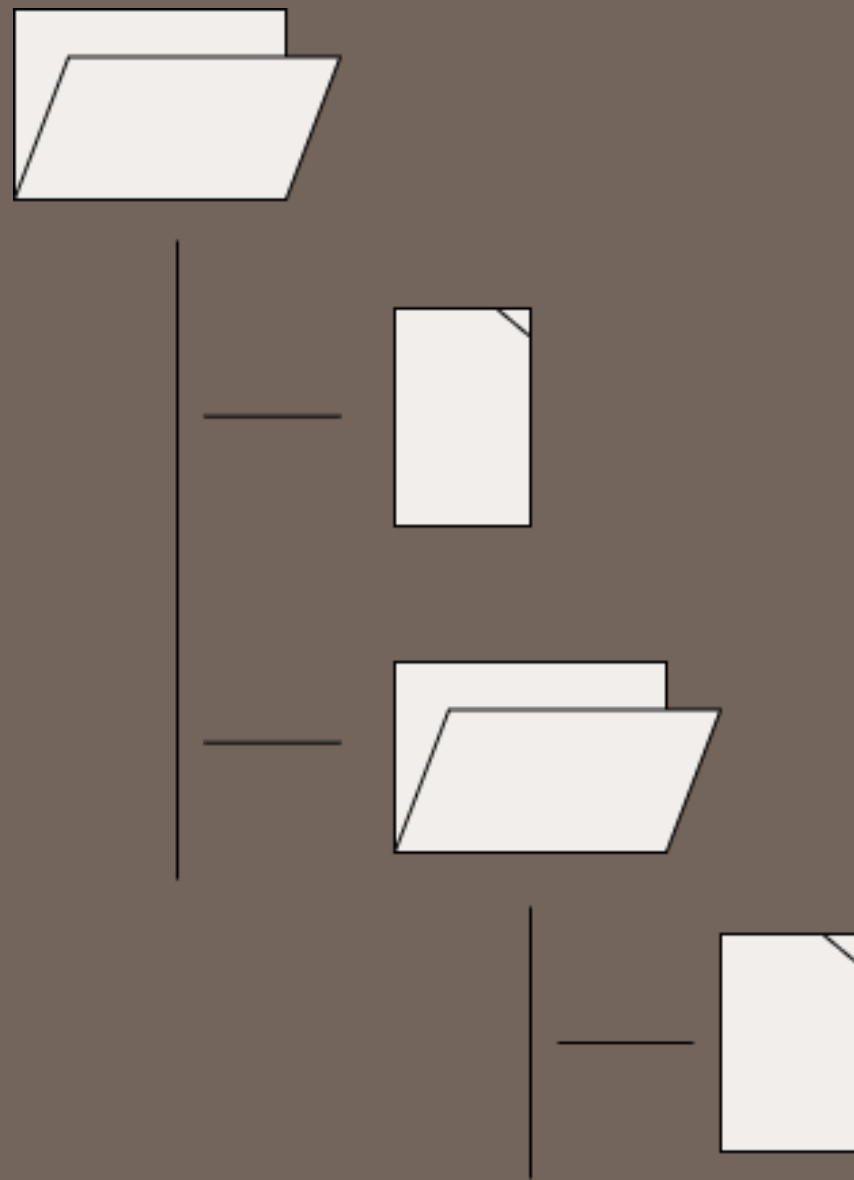
- QU'EST CE QU'UN FILESYSTEM
- CHOIX D'UNE SOLUTION
- DÉTAILS DE NOTRE IMPLÉMENTATION
- CONCLUSION

SOMMAIRE

- QU'EST CE QU'UN FILESYSTEM
- CHOIX D'UNE SOLUTION
- DÉTAILS DE NOTRE IMPLÉMENTATION
- CONCLUSION

DÉFINITION

À quoi sert un filesystem ?



- **ORGANISE, STOCK ET RETROUVE LES DONNÉES**
- **GÈRE ALLOC DES BLOCS, PERMS, METADATA...**
- **DÉPEND DU SUPPORT ET DE L'IMPLÉMENTATION**
- **INTERFACE BAS NIVEAU ENTRE OS ET STOCKAGE**

EXEMPLES

FAT	EXT	BTRFS
SIMPLE, ANCIEN ET TRÈS COMPATIBLE	STANDARD LINUX HISTORIQUE, BASÉ SUR INODES	MODERNE, CONÇU POUR LA RÉSILIENCE ET LES FONCTS. AVANCÉES
PEU DE METADATAS, ET PAS DE JOURNALISATION	JOURNALISATION (EXT3/4), EXTENTS, HAUTE STABILITÉ	C-O-, SNAPSHOTS, SUBVOLUMES, CHECKSUMS ...
LIMITE DE 4GO PAR FICHIERS, PEU FIABLE EN CAS DE CRASH	USAGE GÉNÉRAL LINUX, PERFORMANT ET ÉPROUVÉ	SERVEURS, STOCKAGE AVANCÉ, BESOIN D'INTÉGRITÉ DES DONNÉES

VIRTUAL FS



VIRTUAL FS

Tâche 1

Tâche 2

RÉALISE DES **APPELS**
SYSTÈMES POUR
MANIPULER LES FICHIERS

VFS

- API UNIFIÉE
- CACHING
- SÉCURITÉ GLOBALE
- ARBORESCENCE GLOBALE
- RÉOLUTION DES CHEMINS
- FS DISTANTS/VIRTUELS

EXT4

FAT32

NTFS

SD

USB

- ORGANISE LA DONNÉE
- STOCK SUR LE SUPPORT
- FOURNIT DES **FONCTIONS DE MANIPULATION** BASIQUES
- **S'ENREGISTRE** AUPRÈS DU VFS

SOMMAIRE

- QU'EST CE QU'UN FILESYSTEM
- CHOIX D'UNE SOLUTION
- DÉTAILS DE NOTRE IMPLÉMENTATION
- CONCLUSION

RAM FS

Définition et rôle

un RAM FS est un système de fichiers entièrement stocké en mémoire vive

accès extrêmement rapide

Volatile
(non persistant)

Simple à implémenter

RAM FS

Exemples

tmpfs

- FileSystem en RAM géré par le noyau
 - Taille limitée
 - Les pages peuvent être libérées si la RAM manque
-

ramfs

- FileSystem en RAM très bas niveau
 - Aucune limite de taille
 - Ne libère jamais la mémoire
-

initramfs

- RAM FS utilisé uniquement au démarrage
 - Chargé par le bootloader avant le vrai disque
 - Contient les outils pour monter le vrai filesystem
-

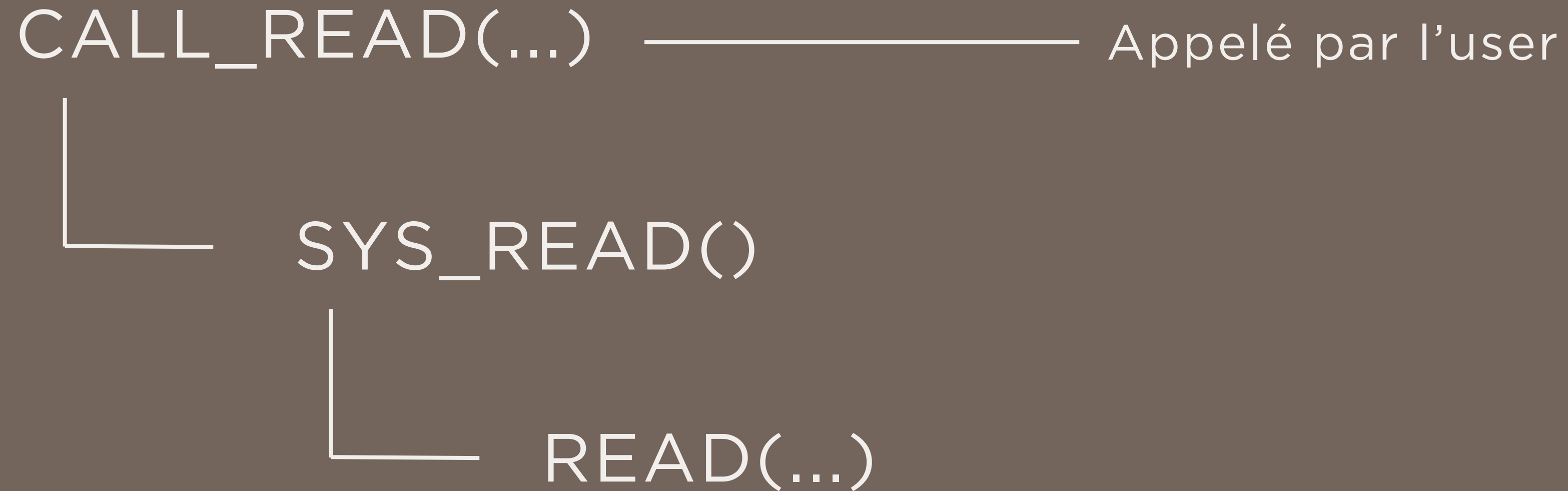
SOMMAIRE

- QU'EST CE QU'UN FILESYSTEM
- CHOIX D'UNE SOLUTION
- DÉTAILS DE NOTRE IMPLÉMENTATION
- CONCLUSION

SYSTEM CALL



SYSTEM CALL



SYSTEM CALL

CALL_READ(...)



SYS_READ()



READ(...)

```
.macro make_syscall name, num
.global \name:
    mov    w8, #\num
    svc    #0
    ret
.endm
```

SYSTEM CALL

CALL_READ(...)



SYS_READ()



READ(...)



real
implementation

API VFS

DESCRIPTEUR
DE FICHIER

fd_t

Type manipulé par
l'utilisateur, représente
un fichier ouvert

ENSEMBLE DE
FONCTIONS

create_file
open_file

...

Fonctions appelées par
les syscall

CODES
D'ERREUR

VFS_OK
VFS_INVALID_FD

...

Codes d'erreur
retournés par les
fonctions

IMPLÉMENTATION VFS

LECTURE DES
CHEMINS

`parse_path`

REPRÉSENTATION
DES FICHIERS
`file_descriptor_t`

SUPPRESSION
RÉCURSIVE
`delete_inode`

TABLEAU DES
FICHIERS OUVERTS
`open_inodes`

ITÉRATEUR SUR LES
DOSSIERS
`get_next_dir_son`

VFS CONNAÎT LE FS
`#include <filesystem.h>`

IMPLÉMENTATION FS

IMPLEMENTATION DU RAM FS

RAM_inode

NOMBRE STATIQUE D'INODES

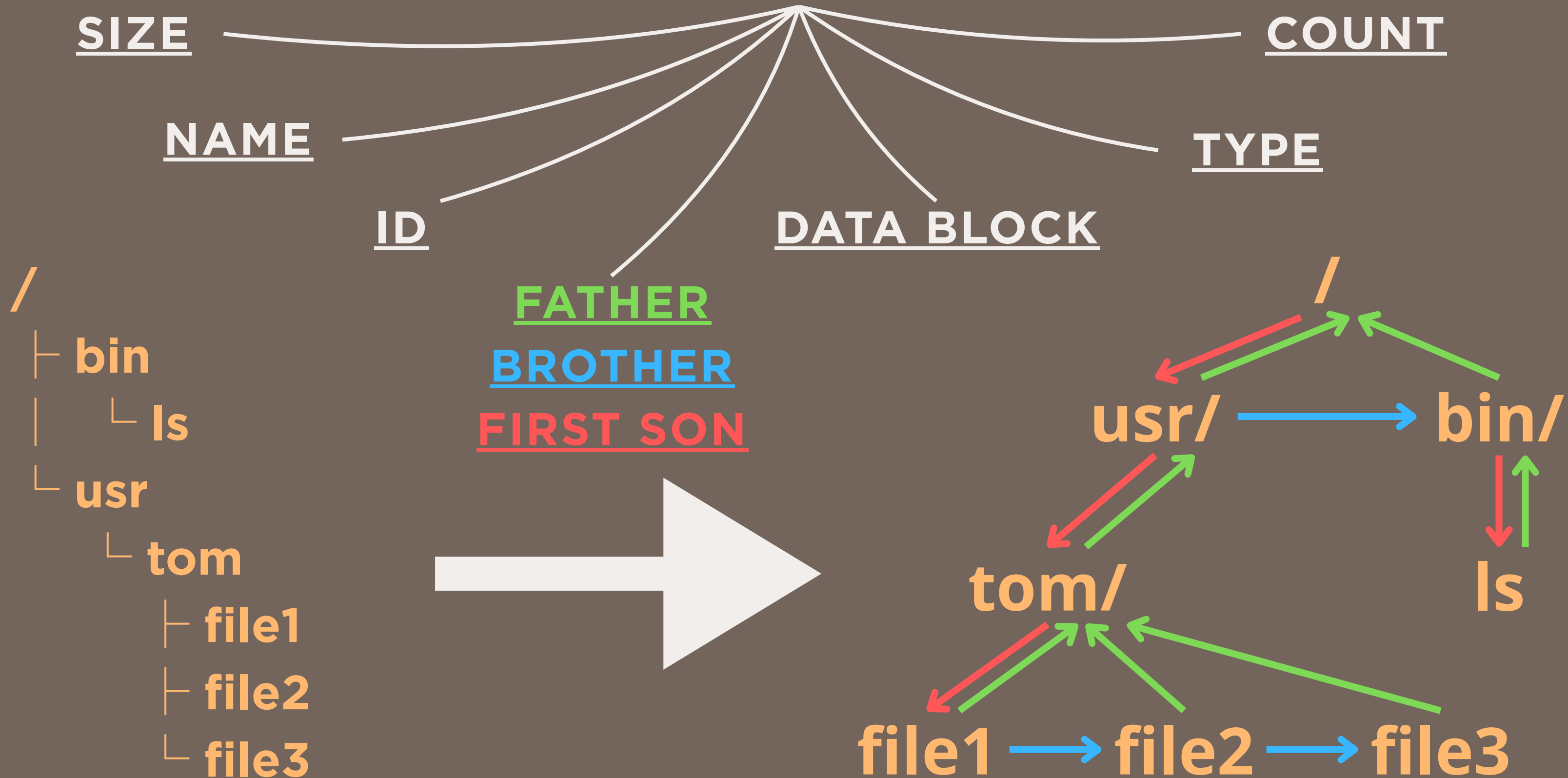
FS_NPAGES

ENSEMBLE DE FONCTIONS

alloc_inode
free_inode
reset_inode

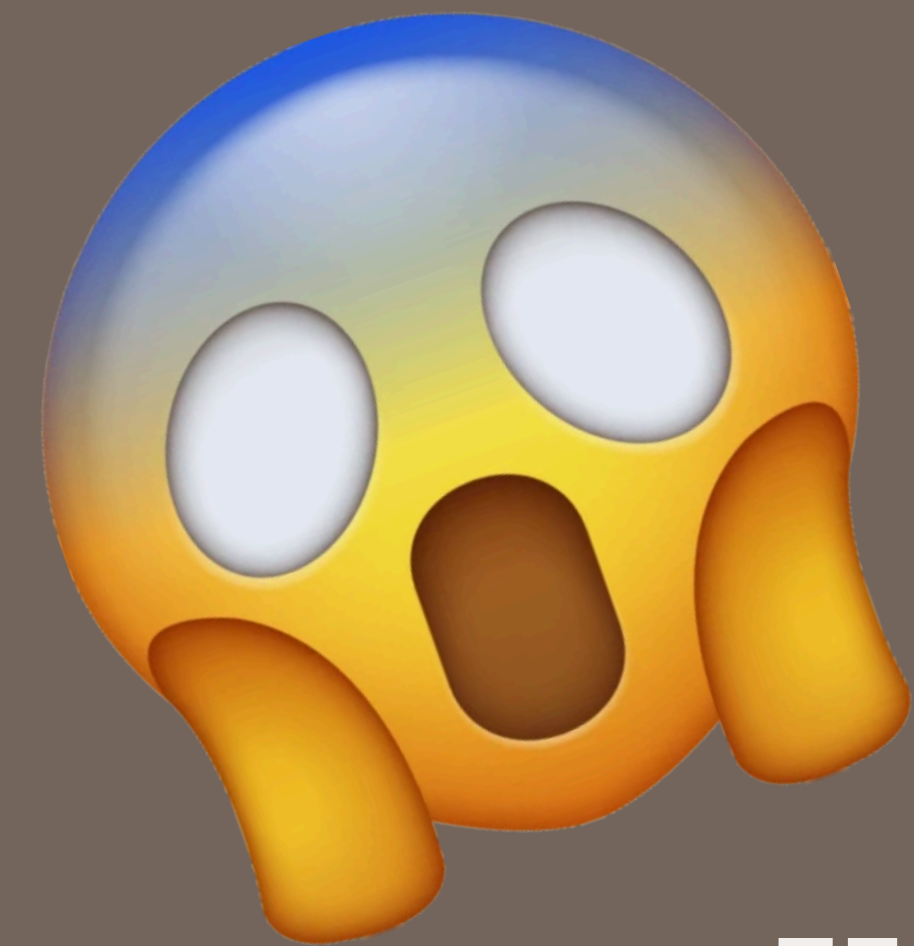
read_inode
write_inode
init_fs

INODES



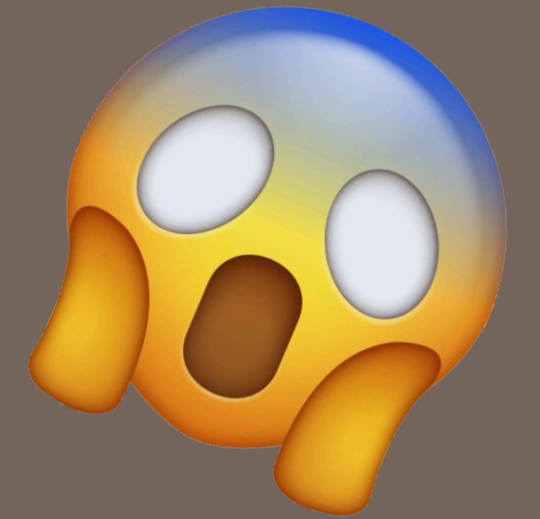
SOMMAIRE

- QU'EST CE QU'UN FILESYSTEM
- CHOIX D'UNE SOLUTION
- DÉTAILS DE NOTRE IMPLÉMENTATION
- CONCLUSION



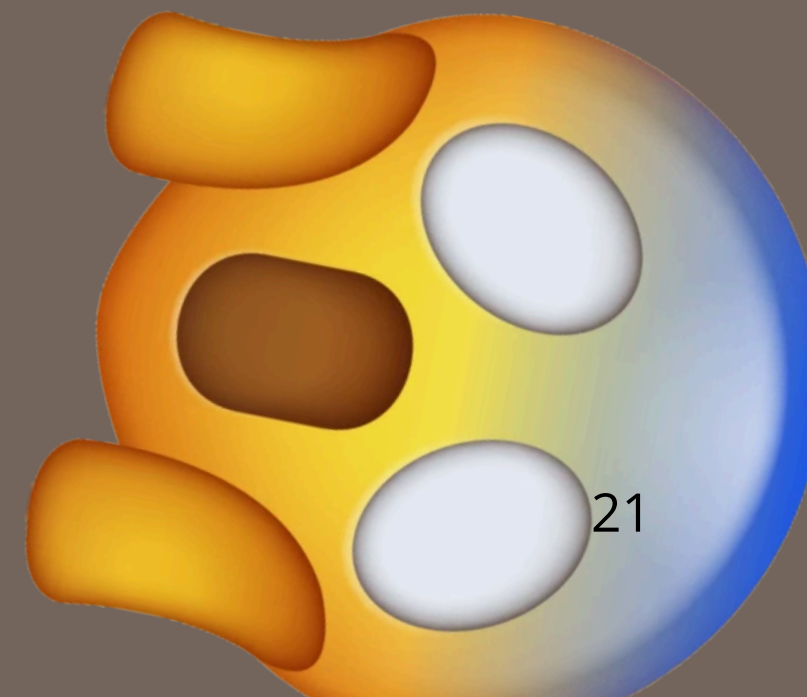
ÇA MARCHE !

même sur Raspberry pi 3 !!



TESTS : PROCESSUS UTILISATEUR

- Création et suppression de fichiers / dossiers
- Écriture / lecture / ajout dans un fichier
- Parcours d'un dossier
- Gestion d'une arborescence



Output Serial Monitor X

Message (Enter to send message to 'Arduino Uno' on 'COM6')

```
kernel boots ...
FS init OK
Vector init OK
Timer init OK
Interrupt init OK
IRQ enable OK
Kernel process started. EL 1
User process, starting the call to the VFS :
Test create file </>:<file1.txt> : OK
Test open file </file1.txt> : OK
Test write <abc123> in file : OK
Test read file : OK (abc123)
Test close file : OK
Test open file </file1.txt> : OK
Test write <abc123> in file : OK
Test read file : OK (abc123abc123)
Test close file : OK
Test create dir </>:<src> : OK
Test create file </src/>:<file2.txt> : OK
Test create file </src/>:<file3.txt> : OK
Test open dir </src/> : OK
Test get next son : OK (file2.txt)
Test get next son : OK (file3.txt)
Test get next son : OK ()
Test close dir : OK
Test delete file </src/file2.txt> : OK
Test open dir </src/> : OK
Test get next son : OK (file3.txt)
Test get next son : OK ()
Test close dir : OK
Test delete file </src/> : OK
Test open dir </> : OK
Test get next son : OK (file1.txt)
Test get next son : OK ()
Test close dir : OK
```

THE END.....

Un grand merci
POUR VOTRE ATTENTION !

RÉFÉRENCES :

[HTTPS://WIKI.OSDEV.ORG/INTRODUCTION](https://wiki.osdev.org/Introduction)

[HTTPS://GITHUB.COM/MPOQUET/RASPBERRY-PI-OS-QEMU](https://github.com/mpoquet/raspberry-pi-os-qemu)

[HTTPS://GITHUB.COM/MIT-PDOS/XV6-RISCV](https://github.com/mit-pdos/xv6-riscv)

[HTTPS://GITHUB.COM/UGO-VALLAT/CLB-FILESYSTEM.GIT](https://github.com/ugo-vallat/clb-filesystem.git)

[HTTPS://PEKOPEK011.SAKURA.NE.JP/UNIX_V6/XV6-BOOK/EN/FILE_SYSTEM.HTML](https://pekopek011.sakura.ne.jp/unix_v6/xv6-book/en/file_system.html)